

SpaceMolt Commander — Game Sequences

Vordefinierte Aktionssequenzen für häufige Spielabläufe. Anstatt dass das LLM jeden Schritt einzeln orchestriert, wählt es einfach eine Sequenz aus. Die Ausführung erfolgt deterministisch in Python — schneller, zuverlässiger, und **60-80% weniger Token**.

Verfügbare Sequenzen

fly_to_station

Beschreibung: Fliege zu einer Station und docke an. Optional mit Systemsprung.

Parameter	Typ	Pflicht	Beschreibung
station_id	string	✓	Zielstation-ID
system_id	string	✗	Springe zuerst in dieses System

Schritte: [jump →] travel → dock

Tick-Kosten: 2-3

Beispiel:

```
{"sequence": "fly_to_station", "params": {"station_id": "ST-42", "system_id": "Sol"}}}
```

mine_and_return

Beschreibung: Abdocken, zum Asteroidenfeld fliegen, scannen, N-mal minen, zurück zur Station.

Parameter	Typ	Pflicht	Beschreibung
asteroid_field	string	✓	Asteroidenfeld POI-ID
home_station	string	✓	Rückkehr-Station
mine_cycles	integer	✗	Mining-Zyklen (Standard: 3)

Schritte: undock → travel → scan → mine(×N) → travel back → dock

Tick-Kosten: 5-8

Beispiel:

```
{"sequence": "mine_and_return", "params": {"asteroid_field": "AF-7", "home_station": "ST-3", "mine_cycles": 5}}}
```

trade_route

Beschreibung: Komplette Handelsroute: zur Kaufstation fliegen, Ware kaufen, zur Verkaufsstation fliegen, verkaufen.

Parameter	Typ	Pflicht	Beschreibung
buy_station	string	✓	Kaufstation
sell_station	string	✓	Verkaufsstation
commodity	string	✓	Handelsware
quantity	integer	✗	Menge (Standard: max leistbar)
buy_system	string	✗	System der Kaufstation
sell_system	string	✗	System der Verkaufsstation

Schritte: fly_to_station(buy) → buy → undock → fly_to_station(sell) → sell

Tick-Kosten: 6-10

Beispiel:

```
{
  "sequence": "trade_route",
  "params": {
    "buy_station": "ST-1",
    "sell_station": "ST-5",
    "commodity": "Iron Ore"
  }
}
```

combat_patrol

Beschreibung: Kampfzone patrouillieren: fliegen, scannen, Gegner angreifen.

Parameter	Typ	Pflicht	Beschreibung
zone	string	✓	Kampfzonen POI-ID
max_engagements	integer	✗	Max. Gegner (Standard: 3)
stance	string	✗	Kampfhaltung: fire/evade/brace/flee (Standard: fire)

Schritte: undock → travel → scan → (stance + attack) × N

Tick-Kosten: 5-10

Beispiel:

```
{"sequence": "combat_patrol", "params": {"zone": "CZ-12", "stance": "fire", "max_engagements": 5}}
```

repair_and_refuel

Beschreibung: Zur Station fliegen, Schiff reparieren, auftanken.

Parameter	Typ	Pflicht	Beschreibung
station	string	✓	Stations-ID
system_id	string	✗	Springe zuerst in dieses System

Schritte: fly_to_station → repair → refuel

Tick-Kosten: 4-5

Beispiel:

```
{"sequence": "repair_and_refuel", "params": {"station": "ST-3"}}
```

sell_all_cargo

Beschreibung: Gesamte Ladung an aktueller oder angegebener Station verkaufen.

Parameter	Typ	Pflicht	Beschreibung
station	string	✗	Station (optional, bei Angabe wird dorthin geflogen)

Schritte: [fly_to_station →] get_cargo → sell all

Tick-Kosten: 1-3

Beispiel:

```
{"sequence": "sell_all_cargo", "params": {}}
```

explore_system

Beschreibung: System erkunden: Info, Position, Scan. Alle Abfragen sind kostenlos.

Parameter	Typ	Pflicht	Beschreibung
system_id	string	✗	System (optional, sonst aktuelles System)

Schritte: [jump →] get_system → get_location → scan

Tick-Kosten: 0-1

Beispiel:

```
{"sequence": "explore_system", "params": {"system_id": "Alpha-Centauri"}}
```

accept_and_track_mission

Beschreibung: Missionen auflisten, optional eine annehmen, Status prüfen.

Parameter	Typ	Pflicht	Beschreibung
mission_id	string	✗	Mission-ID (ohne = nur auflisten)

Schritte: get_missions → [accept_mission →] get_status

Tick-Kosten: 0-1

Beispiel:

```
{"sequence": "accept_and_track_mission", "params": {"mission_id": "M-15"}}
```

full_status_check

Beschreibung: Umfassende Statusprüfung: Status, Position, Ladung, Schiff, Fähigkeiten. Alle Abfragen KOSTENLOS.

Parameter	Typ	Pflicht	Beschreibung
(keine)	—	—	—

Schritte: get_status → get_location → get_cargo → get_ship → get_skills

Tick-Kosten: 0

Beispiel:

```
{"sequence": "full_status_check", "params": {}}
```

Token-Einsparungen

Vergleich: Sequenz vs. Einzelaufrufe

Workflow	Einzelaufrufe (Token)	Sequenz (Token)	Einsparung
Zur Station fliegen & docken	~800 (3 tool calls + Entscheidungen)	~200 (1 tool call)	75%
Mining-Zyklus (3x)	~2.500 (8 tool calls + Planung)	~300 (1 tool call)	88%
Handelsroute	~3.000 (10+ tool calls)	~350 (1 tool call)	88%
Kampfpatrouille	~2.000 (8+ tool calls)	~300 (1 tool call)	85%
Voller Statuscheck	~1.500 (5 tool calls)	~200 (1 tool call)	87%

Warum Token sparen wichtig ist:

1. **Kosten:** Weniger API-Aufrufe = günstigere LLM-Nutzung
2. **Geschwindigkeit:** Eine Sequenz läuft in Sekunden, statt mehrere LLM-Runden abzuwarten
3. **Zuverlässigkeit:** Deterministische Python-Logik statt LLM-Entscheidungen für Routineaufgaben
4. **Context Window:** Mehr Platz für strategische Entscheidungen statt Routine-Orchestrierung

Wann welche Methode?

Situation	Methode
Routine-Aktion (fliegen, minen, handeln)	<code>execute_sequence</code>
Komplexe Strategie-Entscheidung	<code>game</code> (Einzelaufrufe) + LLM-Planung
Datenanalyse (Marktvergleich, Route berechnen)	<code>execute_code</code>
Unbekannte/neue Spielsituation	<code>game</code> (Einzelaufrufe)

Architektur

```

LLM entscheidet: "Ich muss minen und zurückkehren"
|
▼
execute_sequence(sequence="mine_and_return", params={...})
|
▼
game_sequences.py:mine_and_return()
├─ api.execute("spacemolt/undock")
├─ api.execute("spacemolt/travel", {...})
├─ api.execute("spacemolt/scan")
├─ api.execute("spacemolt/mine") × N
├─ api.execute("spacemolt/travel", {...})
└─ api.execute("spacemolt/dock", {...})
|
▼
Ergebnis: "✅ SUCCESS (8/8 steps) – Mined 3/3 cycles, returned to station"

```

Das LLM entscheidet **WAS** (welche Sequenz), Python entscheidet **WIE** (Ausführung).